

Comenzado el	viernes, 22 de noviembre de 2024, 08:56
Estado	Finalizado
Finalizado en	viernes, 22 de noviembre de 2024, 09:02
Tiempo empleado	6 minutos 14 segundos
Puntos	18/20
Calificación	9 de 10 (89%)

Pregunta **1**

Incorrecta

Se puntúa 0 sobre 1

Suponga que la versión recursiva de la función para el cálculo del factorial que se presentó en clases fuese redefinida en la forma siguiente:

```
def factorial(n):  
    f = n * factorial(n - 1)  
    if n == 0:  
        return 1  
    return f
```

¿Cuál es el problema (si lo hubiese) con esta versión de la función?

Seleccione una:

- ☐ a. La función siempre retorna un 1, sin importar cual sea el valor de n.
- ☒ b. No hay ningún problema. **✗ Incorrecto...** cada vez que la función comienza a ejecutarse, produce de inmediato otra llamada recursiva sin permitir nunca que se llegue al if o al return final... Funciona correctamente.
- ☐ c. Para calcular el factorial necesita un ciclo for que genere los números a multiplicar. Así planteada, la función hace un único primer cálculo incompleto y lo retorna.
- ☐ d. Contiene todos los elementos que debería tener una función recursiva bien planteada, pero en el orden incorrecto: la condición para chequear si n es cero debería estar antes que la llamada recursiva para evitar la recursión infinita.

Revise la Ficha 26, página 537 y siguientes.

La respuesta correcta es:

Contiene todos los elementos que debería tener una función recursiva bien planteada, pero en el orden incorrecto: la condición para chequear si n es cero debería estar antes que la llamada recursiva para evitar la recursión infinita.

Pregunta **2**

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes son ciertas respecto de la estrategia de resolución de problemas conocida como *Algoritmos Ávidos*?

Seleccione una o más de una:

- ☐ a. Se trata de un planteo basado en explorar y analizar cada camino posible para resolver un problema, de forma que si detecta que algún camino conduce a una solución incorrecta, se abandone ese camino y se regrese para explorar otros posibles, hasta dar con la solución correcta.
- ☒ b. Se trata de un planteo basado en identificar una regla local que intuitivamente parece correcta, y aplicarla una y otra vez sin volver atrás ni analizar caminos alternativos, hasta llegar a la solución del problema global. ✓ ¡Correcto!
- ☒ c. Normalmente, una ventaja de un algoritmo ávido es que, si la regla local aplicada es correcta, suele ser sencillo de plantear y de comprender, además de veloz para ejecutar. ✓ ¡Correcto!
- ☐ d. Normalmente, una ventaja de intentar aplicar un algoritmo ávido es que si bien la validez de la regla local debe ser demostrada formalmente, eso es comúnmente fácil de hacer.

¡Correcto!

Las respuestas correctas son:

Se trata de un planteo basado en identificar una regla local que intuitivamente parece correcta, y aplicarla una y otra vez sin volver atrás ni analizar caminos alternativos, hasta llegar a la solución del problema global.,

Normalmente, una ventaja de un algoritmo ávido es que, si la regla local aplicada es correcta, suele ser sencillo de plantear y de comprender, además de veloz para ejecutar.

Pregunta **3**

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes expresiones describe la forma de trabajo general de una *Pila*?

Seleccione una:

- ☐ a. OIFO (Ordered In - First Out)
- ☒ b. LIFO (Last In - First Out) ✓ ¡Correcto!
- ☐ c. FIFO (First In - First Out)
- ☐ d. OIRO (Ordered In - Random Out)

¡Correcto!

La respuesta correcta es:

LIFO (Last In - First Out)

Pregunta **4**

Correcta

Se puntúa 1 sobre 1

¿Cuál es la razón por la que es recomendable usar una pila como soporte al recorrer un árbol binario (y por lo tanto, cuál es la razón por la que es recomendable usar recursión para ese recorrido)?

Seleccione una:

- ☒ a. Porque una pila permite almacenar en orden LIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el tope de la pila y será accesible en forma directa. ✓ ¡Correcto!
- ☐ b. No es necesario ni usar una pila ni usar recursión. El árbol puede recorrerse simplemente con un ciclo que comience en la raíz y luego se desplace vuelta tras vuelta hacia el hijo izquierdo, hasta llegar a un hijo izquierdo nulo, momento en el cual el ciclo debe detenerse y el árbol queda completamente recorrido.
- ☐ c. No hay ninguna razón para usar una pila o para usar recursión. Un ciclo y una referencia de persecución serían siempre suficientes para hacer el recorrido completo.
- ☐ d. En realidad, es mejor usa una cola en lugar de una pila. Una cola permite almacenar en orden FIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el frente de la cola y será accesible en forma directa.

¡Correcto!

La respuesta correcta es:

Porque una pila permite almacenar en orden LIFO las direcciones de los nodos que se acaban de visitar en el recorrido, y cuando se necesite regresar a ese nodo luego de procesar alguno de sus dos hijos, su dirección quedará en el tope de la pila y será accesible en forma directa.

Pregunta **5**

Correcta

Se puntúa 1 sobre 1

¿Qué se entiende, en el contexto del Análisis de Algoritmos, por un *Orden de Complejidad*?

Seleccione una:

- ☒ a. Un conjunto o familia de funciones matemáticas que se comportan asintóticamente de la misma forma. ✓ ¡Correcto!
- ☐ b. Un conjunto o familia de algoritmos que resuelven el mismo problema.
- ☐ c. Un conjunto o familia de subrutinas con similares objetivos (equivalente al concepto de *módulo*).
- ☐ d. Un conjunto de datos ordenados.

¡Correcto!

Las respuestas correctas son:

Un conjunto o familia de funciones matemáticas que se comportan asintóticamente de la misma forma.,

Un conjunto o familia de algoritmos que resuelven el mismo problema.

Pregunta 6

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes describe mejor la idea de funcionamiento en la que está basado el algoritmo conocido como ordenamiento por *Intercambio Directo* u *Ordenamiento de Burbuja* para ordenar *de menor a mayor* un arreglo de n componentes?

Seleccione una:

- ☒ a. Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vayan acomodándose al final del arreglo. ✓ ¡Correcto!
- ☐ b. Suponer que el arreglo tiene un subconjunto inicialmente ordenado que contiene sólo al primer elemento, luego realizar n pasadas, de forma que en cada una agregue el siguiente elemento al grupo que está ordenado.
- ☐ c. Realizar n pasadas, de forma que en cada una se determine el menor de los elementos analizados, y llevar ese menor a la casilla pivot.
- ☐ d. Reacomodar los n elementos del arreglo en forma aleatoria, controlar si quedó ordenado, y en caso negativo, volver a reacomodarlos en forma aleatoria, continuando así hasta que en algún momento se obtenga un arreglo ordenado...

¡Correcto!

La respuesta correcta es:

Realizar n pasadas, de forma que en cada una se compare a cada elemento con el siguiente, logrando que en cada pasada los mayores vayan acomodándose al final del arreglo.

Pregunta 7

Correcta

Se puntúa 1 sobre 1

A lo largo del curso hemos visto la forma de plantear programas controlados por menú de opciones, y sabemos que esos programas incluyen un ciclo para el control del proceso. Pero también podríamos hacer un planteo basado en recursión, sin usar el ciclo, en forma similar a la que se muestra aquí:

```
_author__ = 'Cátedra de AED'

def menu():
    print('1. Opcion 1')
    print('2. Opcion 2')
    print('3. Salir')
    op = int(input('Ingrese opcion: '))
    if op == 1:
        print('Eligio la opcion 1...')
    elif op == 2:
        print('Eligio la opcion 2...')
    elif op == 3:
        return
    menu()

# script principal...
menu()
```

¿Hay algún problema o contraindicación respecto del uso y aplicación de esta versión recursiva para la gestión de un menú? Seleccione la respuesta que mejor describa este punto.

Seleccione una:

- ☐ a. La versión recursiva que se mostró no funciona. Sólo permite cargar una vez la opción elegida, e inmediatamente se detiene.
- ☒ b. Esta versión recursiva posiblemente sea más simple y compacta que la versión basada en un ciclo, pero la versión recursiva pide memoria de stack cada vez que se invoca, por lo que es más conveniente la iterativa. ¡Correcto! De todos modos, si la idea es usar el menú sólo para un par de operaciones y terminar de inmediato, no habrá problema alguno. Lo malo sería que el programa funcione en forma permanente, las 24 horas del día y todos los días... pues en ese caso, se corre el serio riesgo de provocar un overflow de stack.
- ☐ c. Esta versión recursiva es completamente equivalente a la versión iterativa. No hay ningún motivo para preferir la una sobre la otra. Da lo mismo si el programador usa la recursiva o la iterativa.
- ☐ d. No hay ningún problema ni contraindicación. Y no sólo eso: la versión recursiva es más simple y compacta que la versión basada en un ciclo, por lo cual es incluso preferible usarla.

¡Correcto!

La respuesta correcta es:

Esta versión recursiva posiblemente sea más simple y compacta que la versión basada en un ciclo, pero la versión recursiva pide memoria de stack cada vez que se invoca, por lo que es más conveniente la iterativa.

Pregunta **8**

Correcta

Se puntúa 1 sobre 1

Un *grafo simétrico* es un grafo dirigido en el cual se cumple que si existe el arco (x, y) entonces existe también el arco (y, x) (es decir, toda vez que existe un arco en un sentido, existe también el arco en sentido contrario). Suponga un grafo G dirigido y simétrico con n vértices, implementado en forma matricial y asumiendo que la matriz de adyacencias almacena valores *boolean* en forma directa. ¿Qué se puede afirmar que es **verdadero** respecto de la matriz de adyacencias de G ?

Seleccione una:

- ☒ a. La matriz es simétrica (o espejada) respecto de la diagonal principal: el valor en el casillero $ady[x][y]$ siempre será  ¡Correcto!
igual al valor en $ady[y][x]$.
- ☐ b. Todos los casilleros del triángulo inferior de la matriz de adyacencia valen *True* y los del triángulo superior valen *False*.
- ☐ c. Todos los casilleros de la diagonal principal de la matriz de adyacencia valen *True*.
- ☐ d. Todos los casilleros de la matriz son *True* o todos los casilleros son *False*.

¡Correcto!

La respuesta correcta es:

La matriz es simétrica (o espejada) respecto de la diagonal principal: el valor en el casillero $ady[x][y]$ siempre será igual al valor en $ady[y][x]$.

Pregunta 9

Correcta

Se puntúa 1 sobre 1

Suponga que se tiene una pila p con capacidad para almacenar números enteros y que las operaciones básicas $pop()$, $peek()$ y $push()$ están correctamente implementadas en el módulo `stack.py` presentado en clases. Suponga que se han insertado los siguientes valores: [8 - 6 - 5 - 3 - 7] (el número 8 es el valor del frente o tope de la Pila). ¿Cuál de las siguientes secuencias de instrucciones permite retirar el valor 5 de la pila, pero dejando el 6 y 8 nuevamente arriba? (es decir: ¿cuál de las siguientes secuencias, dejaría la pila p en el estado [8 - 6 - 3 - 7]?)

Seleccione una:

- ☐ a. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.peak()`
`p.push(n2)`
`p.push(n1)`
- ☒ b. `n1 = p.pop()` ✓ ¡Correcto!
`n2 = p.pop()`
`x = p.pop()`
`p.push(n2)`
`p.push(n1)`
- ☐ c. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.pop()`
`p.push(n1)`
`p.push(n2)`
- ☐ d. `n1 = p.pop()`
`n2 = p.pop()`
`x = p.pop()`

¡Correcto!

La respuesta correcta es:

```
n1 = p.pop()
n2 = p.pop()
x = p.pop()
p.push(n2)
p.push(n1)
```

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

¿Cuáles de las siguientes afirmaciones **son ciertas** en referencia a las *Estrategias de Resolución de Problemas* que se citan? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero  ¡Correcto!
normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado.
- ☐ b. El empleo de la *Recursividad* para resolver un problema no es recomendable en ningún caso, debido a la gran cantidad de recursos de memoria o de tiempo de ejecución que implica.
- ☒ c. La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error  ¡Correcto!
explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.
- ☒ d. La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o  ¡Correcto!
tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.

¡Correcto!

Las respuestas correctas son:

La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado.,

La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.,

La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.

Pregunta **11**

Parcialmente correcta

Se puntúa 0 sobre 1

¿Cuáles de las siguientes afirmaciones **son ciertas** en relación a conceptos asociados con la **recursividad**?

Seleccione una o más de una:

- ☐ a. Si una función es recursiva, entonces no debe incluir ningún ciclo en su bloque de acciones.
- ☒ b. Una función recursiva no puede incluir **✗ Incorrecto... Puede incluir tantas como necesite. Revise la estrategia Divide y Vencerás, o los distintos algoritmos que hemos analizado para graficar fractales si quiere algún ejemplo...** más de una invocación a si misma en su bloque de acciones.
- ☐ c. Cada instancia recursiva que es ejecutada, almacena dos grupos de datos en el stack segment: la dirección de retorno (a la que se debe regresar cuando termine la ejecución de esa instancia) y las variables locales que esa instancia de la función haya creado.
- ☒ d. A medida que se desarrolla la cascada de invocaciones recursivas, el stack segment se va llenando para darle **✓ ¡Correcto!** soporte a cada instancia recursiva, y luego, cuando una instancia logra finalizar y se produce el proceso de vuelta atrás, el stack segment comienza a vaciarse.

Las respuestas correctas son:

Cada instancia recursiva que es ejecutada, almacena dos grupos de datos en el stack segment: la dirección de retorno (a la que se debe regresar cuando termine la ejecución de esa instancia) y las variables locales que esa instancia de la función haya creado.,

A medida que se desarrolla la cascada de invocaciones recursivas, el stack segment se va llenando para darle soporte a cada instancia recursiva, y luego, cuando una instancia logra finalizar y se produce el proceso de vuelta atrás, el stack segment comienza a vaciarse.

Pregunta **12**

Correcta

Se puntúa 1 sobre 1

¿Por qué es considerada una *mala idea* tomar como pivot al *primer elemento* (o al *último*) de cada partición al implementar el algoritmo *Quicksort*?

Seleccione una:

- ☐ a. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el tamaño n del arreglo fuese muy grande.
- ☒ b. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo **✓ ¡Correcto!** estuviese ya ordenado o casi ordenado.
- ☐ c. No es cierto que sea una mala idea. Ambas alternativas son tan buenas como cualquier otra.
- ☐ d. Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese completamente desordenado.

¡Correcto!

La respuesta correcta es:

Porque de esa forma aumenta el riesgo de caer en el peor caso, o aproximarse al peor caso, si el arreglo estuviese ya ordenado o casi ordenado.

Pregunta **13**

Correcta

Se puntúa 1 sobre 1

¿Cuál es el problema si en el algoritmo *Shellsort* se elige una serie de incrementos decrecientes de la forma { ..., 16, 8, 4, 2, 1 } ?

Seleccione una:

- ☒ a. Los subconjuntos analizados contendrán casi los mismos elementos cuando la distancia usada sea cada vez  ¡Correcto! menor, sin garantías de lograr una buena organización del arreglo antes de la última pasada.
- ☐ b. No sólo no hay ningún problema, sino que esa serie es la mejor posible para el algoritmo Shellsort.
- ☐ c. Ningún problema: esa serie es tan buena como cualquier otra.
- ☐ d. El arreglo no quedará ordenado al final.

¡Correcto!

La respuesta correcta es:

Los subconjuntos analizados contendrán casi los mismos elementos cuando la distancia usada sea cada vez menor, sin garantías de lograr una buena organización del arreglo antes de la última pasada.

Pregunta **14**

Parcialmente correcta

Se puntúa 1 sobre 1

¿En cuáles de las siguientes situaciones sería ideal un *grafo no dirigido*? (Más de una puede ser correcta...)

Seleccione una o más de una:

- ☒ a. Para modelar un plano de rutas entre localidades o ciudades.  ¡Correcto!
- ☐ b. Para modelar un sistema de rutas aéreas entre distintos aeropuertos del mundo.
- ☐ c. Para modelar un "sociograma": un gráfico que expresa en qué forma un grupo de personas se relacionan entre sí.
- ☐ d. Para modelar un plan de estudios de una carrera universitaria.

Las respuestas correctas son:

Para modelar un plano de rutas entre localidades o ciudades.,

Para modelar un sistema de rutas aéreas entre distintos aeropuertos del mundo.

Pregunta **15**

Correcta

Se puntúa 1 sobre 1

Para cada uno de los algoritmos o procesos listados en la columna de la izquierda, seleccione la expresión de orden que mejor describe su tiempo de ejecución en el peor caso.

Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.

$O(n^2)$ (léase: "orden n al cuadrado")



Dadas dos matrices de orden $n \times n$, obtener la matriz producto aplicando el método tradicional.

$O(n^3)$ (léase: "orden n al cubo")



Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.

$O(\log(n))$



Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial.

$O(n)$



Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero $v[k]$ (con $0 \leq k \leq n-1$).

$O(1)$



¡Correcto!

La respuesta correcta es:

Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.

→ $O(n^2)$ (léase: "orden n al cuadrado"),

Dadas dos matrices de orden $n \times n$, obtener la matriz producto aplicando el método tradicional. → $O(n^3)$ (léase: "orden n al cubo"),

Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.

→ $O(\log(n))$,

Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial. → $O(n)$,

Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero $v[k]$ (con $0 \leq k \leq n-1$). → $O(1)$

Pregunta **16**

Correcta

Se puntúa 1 sobre 1

¿Cuál es la cantidad de niveles del *árbol de invocaciones recursivas* que se genera al ejecutar el *Quicksort* para ordenar un arreglo de n elementos, en el **peor caso**? (Es decir: ¿Cuál es la *altura* de ese árbol en ese peor caso?)

Seleccione una:

- ☒ a. `Altura = n` ✓ ¡Correcto!
- ☐ b. `Altura = n * log(n)`
- ☐ c. `Altura = n2`
- ☐ d. `Altura = log(n)`

¡Correcto!

La respuesta correcta es:

`Altura = n`

Pregunta **17**

Correcta

Se puntúa 1 sobre 1

¿Cuántas comparaciones **por nivel** se realizan en un árbol de búsqueda de n nodos cuando se busca una clave? (no importa si la búsqueda llega o no al último nivel del árbol: sólo queremos saber cuántas comparaciones se hacen en aquellos niveles hasta los que llegue la búsqueda)

Seleccione una:

- ☐ a. Comparaciones por nivel: `log(n)`
- ☒ b. Comparaciones por nivel: `1 (una)` ✓ ¡Correcto!
- ☐ c. Comparaciones por nivel: `n2`
- ☐ d. Comparaciones por nivel: `n`

¡Correcto!

La respuesta correcta es: Comparaciones por nivel: `1 (una)`

Pregunta 18

Correcta

Se puntúa 1 sobre 1

Considere el problema del Cambio de Monedas analizado en clases, y la solución mediante un Algoritmo Ávido también presentada en clases ¿Cuáles de las siguientes afirmaciones **son ciertas** en relación al problema y al algoritmo citado? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a. El Algoritmo Ávido sugerido para el Problema del Cambio de Monedas falla si el valor x a cambiar tiene una moneda igual a x en el conjunto de valores nominales: en ese caso, el algoritmo provoca un error de runtime y se interrumpe.
- ☐ b. El Algoritmo Ávido sugerido para el problema del Cambio de Monedas funciona correctamente para cualquier conjunto de valores nominales de monedas, siempre y cuando ese conjunto incluya a la moneda de 1 centavo.
- ☐ c. Si el Problema de Cambio de Monedas no puede resolverse en forma óptima para un conjunto dado de monedas que incluya a la de 1 centavo, mediante el Algoritmo Ávido propuesto, entonces el problema no tiene solución.
- ☒ d. Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no  ¡Correcto! habrá solución posible para muchos valores de cambio.

¡Correcto!

La respuesta correcta es:

Sea cual sea el algoritmo que se emplee, es exigible que exista la moneda de 1 centavo, pues de otro modo no habrá solución posible para muchos valores de cambio.

Pregunta 19

Correcta

Se puntúa 1 sobre 1

Si se realiza un análisis preciso del ordenamiento por *Selección Directa* para un arreglo de n componentes, se llega a la conclusión que ese algoritmo hará $n-1$ pasadas, con $n-1$ comparaciones en la primera, $n-2$ en la segunda, y así sucesivamente reduciendo de a 1 la cantidad de comparaciones hasta hacer sólo una comparación en la última pasada. Por lo tanto, el algoritmo hará *invariablemente* una cantidad total de $\frac{1}{2}(n^2 - n)$ comparaciones. Sabiendo esto, ¿cuáles de las siguientes expresiones son correctas para describir la cantidad de comparaciones que hará el algoritmo, usando distintos tipos de notaciones? (Más de una respuesta puede ser correcta. Marque TODAS las que considere correctas)

Seleccione una o más de una:

- ☒ a. Cantidad de comparaciones: $O(n^2)$  ¡Correcto!
- ☐ b. Cantidad de comparaciones: $o(n^2)$
- ☒ c. Cantidad de comparaciones: $\Theta(n^2)$  ¡Correcto!
- ☒ d. Cantidad de comparaciones: $\Omega(n^2)$  ¡Correcto!

¡Correcto!

Las respuestas correctas son:

Cantidad de comparaciones: $O(n^2)$,

Cantidad de comparaciones: $\Omega(n^2)$,

Cantidad de comparaciones: $\Theta(n^2)$

Pregunta **20**

Correcta

Se puntúa 1 sobre 1

¿Cuál de las siguientes estrategias de obtención del pivot es *la más recomendable* para evitar que el algoritmo *Quicksort* se degrade en su peor caso $O(n^2)$ en cuanto a su tiempo de ejecución?

Seleccione una:

- ☐ a. En cada partición, usar como pivot al valor de la casilla central.
- ☒ b. En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición). ✓ ¡Correcto!
- ☐ c. En cada partición, usar como pivot al valor de la primera casilla.
- ☐ d. En cada partición, usar como pivot al valor de la última casilla.

¡Correcto!

La respuesta correcta es:

En cada partición, obtener el pivot por la mediana de tres (ya sea la mediana entre el primero, el último y el central; o bien la mediana entre tres elementos aleatorios la partición).

[◀ Cuestionario 00 \[Temas: Reglas del Juego\]](#)

Ir a...